Exercise Session 4

IESM Fall 2026-2027

Thibault, Evan, Amina, Alice

October 12, 2026

Course Reminders

- Tomorrow, Tuesday 13.10: mock exam solutions and Q&A session
- Monday 19.10: report for exercise 4 due by 8:00; Class break
- Monday 26.10: lecture (post-HF methods)
- **Tuesday 27.10:** written exam

Exercise 4

The Hartree-Fock procedure in detail (Reminder: you can download these slides from the Exercise page)

🎯 Learning goals

Build and diagonalize the Fock matrix

Understand the steps in a HF SCF calculation

Practice implicit summation in numpy

📖 Chapter in script

Chapter 4 - An introduction to Hartree Fock theory

Appendix C

📚 Resources

Introduction to Hartree-Fock Molecular Orbital Theory by the Sherrill group: slides and videos (part 1, part 2)

Recap of HF Theory

HF method = approximate many-body wavefunction to a single Slater determinant

$$\Psi(\mathbf{r}_1, \mathbf{r}_2, \ldots, \mathbf{r}_n) \approx \Psi_{HF} \equiv \frac{1}{\sqrt{N}} |\phi_1(\mathbf{r}_1)\phi_2(\mathbf{r}_2)\ldots\phi_n(\mathbf{r}_n)|$$

$\rightarrow \Psi_{HF}$ inserted into time-independent Schrödinger equation to find eigenvalue, i.e. $E = \left\langle \Psi_{HF} \middle| \hat{H}_{el} \middle| \Psi_{HF} \right\rangle$. What is found is a **variational solution**, i.e. HF energy is always above true energy.

Recap of SCF Method

HF equations (were derived in detail during Lectures)

$$E_{HF} = \sum_i \left\langle \phi_i \middle| \hat{h} \middle| \phi_i \right\rangle + \frac{1}{2} \sum_{i,j} \left([\phi_i \phi_i | \phi_j \phi_j] - [\phi_i \phi_j | \phi_j \phi_i] \right) \qquad \forall \phi_i$$

where $[\ldots|\ldots]$ integrals contain Coulomb and Exchange operators, whose action on orbital ϕ_i depends on all the other one-electron orbitals ϕ_j. Hence, HF equations have to be solved iteratively until self-consistency (**self consistent field SCF method**)

```
                        Total Energy        Delta E     RMS |[F,P]|

 @DF-UHF iter SAD:    -0.85212229561042   -8.52122e-01   0.00000e+00
 @DF-UHF iter   1:    -1.12493424123815   -2.72812e-01   1.62339e-02 DIIS
 @DF-UHF iter   2:    -1.12669596352817   -1.76172e-03   2.38980e-03 DIIS
 @DF-UHF iter   3:    -1.12673509439903   -3.91309e-05   3.36383e-06 DIIS
 @DF-UHF iter   4:    -1.12673509447661   -7.75795e-11   2.74379e-09 DIIS
Energy and wave function converged.
```

Hartree-Fock Roothaan Equations

- HF equations area set of coupled integro-differential equations to determine the HF molecular one-electron orbitals
- If we represent the orbitals in a basis (of AO-like orbitals), the HF equations transform into matrix equations that were first derived by **Roothaan**

$$\Rightarrow \mathbb{FC} = \mathbb{SCE}$$

- Note: as you will see today, this problem can be recasted in an eigenvalue problem via a basis set transformation

$$\Rightarrow \mathbb{F}'\mathbb{C}' = \mathbb{C}'\mathbb{E}'$$

Building the Fock Matrix

The HF method recasts into a pseudo-eigenvalue problem

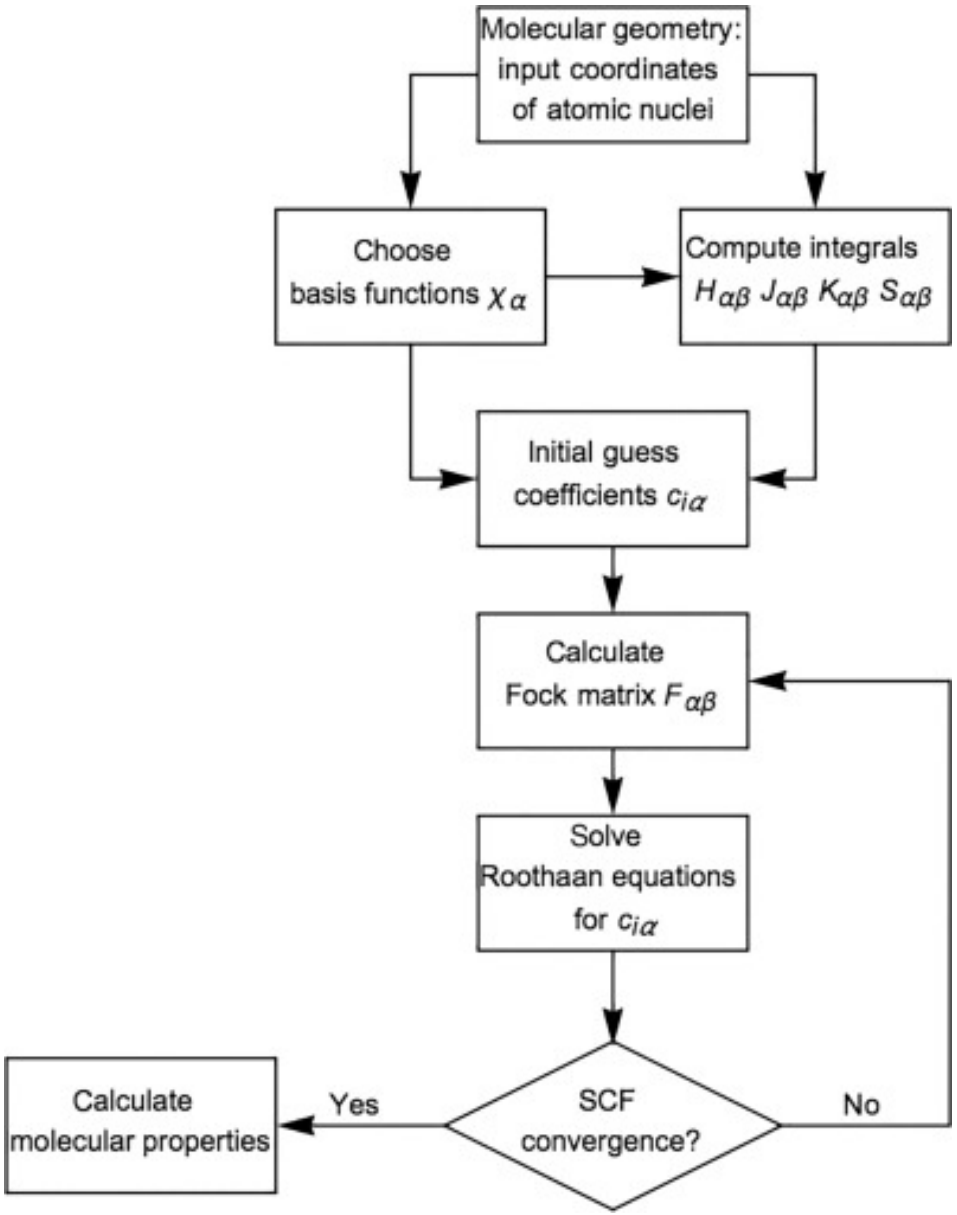
$$\mathbb{FC} = \mathbb{SCE}$$

where:

- $\mathbb{F} = \mathbb{H} + 2\mathbb{J} - \mathbb{K}$ is the **Fock matrix**
- $\mathbb{C}$ is the **wavefunction amplitude matrix**
- $\mathbb{S}$ is the **overlap matrix**
- and $\mathbb{E}$ is the **energy value matrix**

The issue? $\mathbb{F}$ relies on an orbital solution in order to iteratively solve (with **SCF method**) for the "best" molecular orbitals which make a single Slater determinant description of the system wavefunction. We will begin with a guess and iteratively improve on the guess.

Performing HF Explicitly



Overlap Matrix $\mathbb{S}$

$\mathbb{S}$, the overlap matrix, describes the inter-relationships of the basis set vectors. Other details about $\mathbb{S}$:

- The number of basis functions, n, defines the size and shape of $\mathbb{S}$ (n×n)
- $\mathbb{S}$ is an idenity matrix in the case of orthonormal basis set functions
- By properly transforming the $\mathbb{S}$ matrix, we can ensure orthonormality

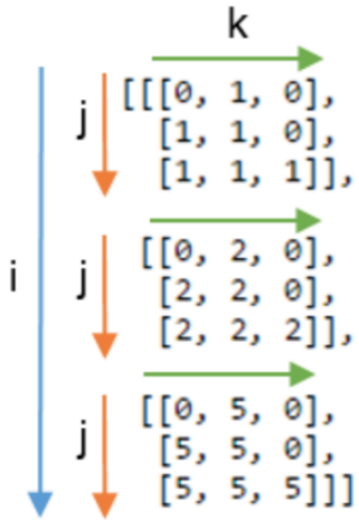
Implicit Einstein Summation

The einsum function in numpy allows for efficient matrix multiplication.

$$(AB)_{pq} = \sum_i A_{p,i} B_{i,q} \qquad \text{explicit summation}$$

$$(AB)_{pq} = A_{p,i} B_{i,q} \qquad \text{implicit summation}$$

```
np.einsum('ij,jk->ijk', A, B)
```




```
np.einsum('ij,jk->ik', A, B)
```

